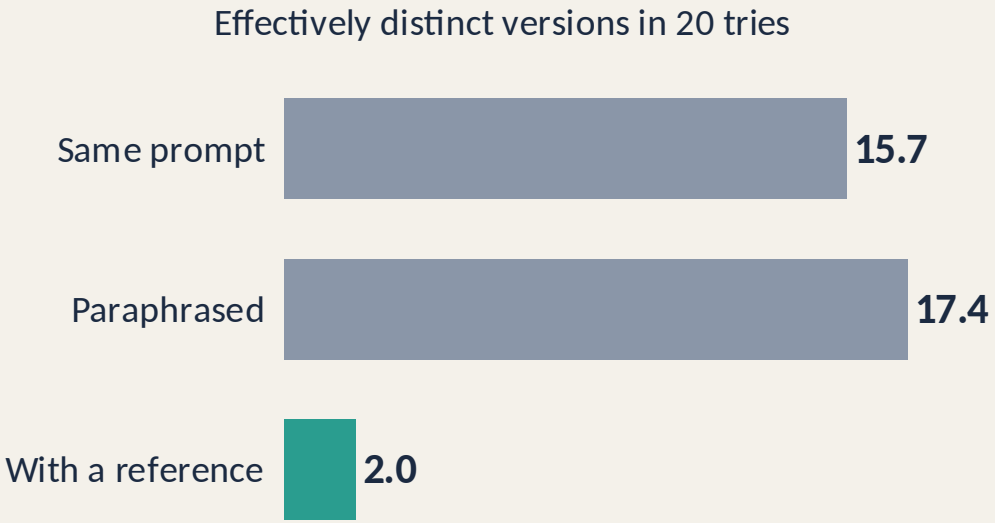
pcat

Reviewed code for AI coding agents: the reviewed version every time, and a human decision whenever it differs.

Evidence from simulations: open-source apps as stand-in organizations, one model, small samples.

Same request, different code

v2.1 · lab · 480 samples



15.7 of 20

effectively distinct versions in 20 tries of the same request

2.0

with a reference in the prompt

Caveat Lab prompts, one model, 8 tasks. The reference was put in the prompt, a perfect lookup.

Why it matters

Reviews repeat

Every new version of an access check, a token helper or a retry loop is new code someone has to review.

Consistency is luck

What you get depends on who asked and how they worded it, not on what your team decided.

Flaws can spread

Inside a repository agents copy what is nearby. A weak version that lands becomes the next example.

Rules files and style guides steer the model. Nothing checks what it actually wrote.

What pcat is

Catalog

Reviewed patterns from your own code, with guards: what reviewers care about. Approved by pull request.

MCP server

Agents search the catalog, fetch the reviewed version and check their code against it.

CI linter

Compares marked code with its pattern. A broken guard fails the pull request until a person decides.

The agent marks what it used: `# pcat: auth.password.pbkdf2@v1`

Engineers review a pattern once; agents reuse it; CI checks the result.

How it works in a pull request

Marker

```
# pcat:
auth.password.
pbkdf2@v1
```

the agent says which pattern it used

Guard

```
must_call:
hmac.
compare_digest
```

the reviewer's rule, kept in the pattern

Deviation

```
verify_password
no longer calls
compare_digest
```

high: the pull request fails

Human decision

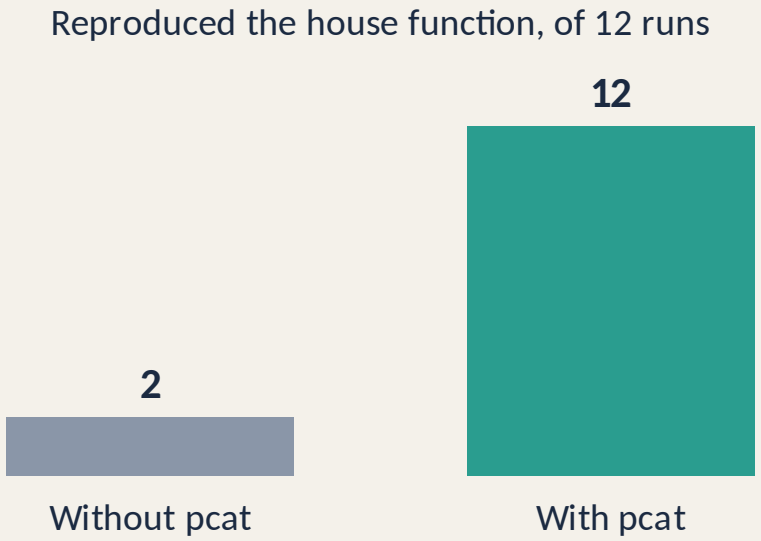
```
# pcat-allow:
PC100 #e96d53e0
-- reason
```

a proposal until an approver confirms it

Tests would still pass: == gives the same answer. The guard catches the property the reviewer cared about.

First-time code: the catalog decides

v5 · 3 apps · 144 runs



On the 4 tasks where the function existed nowhere else in the repo, agents with pcat wrote the house version 12/12 times; without it, 2/12.

Where similar code sits nearby, agents copy it with or without pcat (similarity 1.00 in both).

With pcat: searched 71/72, right pattern 54/54, forced onto unrelated work 0/36.

Caveat Open-source apps as stand-in organizations; catalogs reviewed by the builder; 3 samples per task; one model.

Silent regressions become decisions

v4.1 rules · v5 · v5.1

20 of 60

lab runs broke a guarded property of the reviewed pattern

20 of 20

flagged, each with a hashed approval proposal for a person to confirm

0

went through without a human decision

0/72 With markers, no correct code failed CI in 144 runs inside the three apps (an earlier linter version failed 5, all explained and fixed).

Caveat One rule set. The lab requests asked for the deviation ("not on 4xx errors"; 429 is a 4xx). Agents write the proposals; a human must still read them.

Supporting: a flawed neighbour

v5.2 · 108 runs · small

Runs that inherited a planted flaw, of 54



All 3 were in the 2 tasks where the flawed function was the obvious stand-in: 3/6 vs 0/6.

51/54 runs without the catalog did not copy the flaw.

Default CI caught 0/3: copied code has no marker.

Caveat We planted the flaws; 6 runs carry the effect; self-reviewed catalogs; one model. Supporting only.

What pcat is not

Not a style guide

It checks the properties reviewers care about (guards), not formatting or naming.

Not a code generator

The agent still writes the code. pcat supplies reviewed examples and checks the result.

Not proven at scale

No real team has used it yet: 3 open-source apps, one model, small samples.

Noisy where unmarked: of 90 sampled findings on code nobody marked, 75 were false positives and 0 true. They stay advisory, out of required checks.

How to try it

```
pipx install "pcat[mcp] @ git+…/pcat"

cd your-repo
pcat init --starter          # dry run
pcat init --starter --apply

pcat audit . --out pcat-audit
```

- About 30 minutes with docs/getting-started.md
- 7 starter patterns; add your own from .pcat/candidates.md
- Claude Code finds the server through .mcp.json
- Two pull-request checks; the audit is one-time, never a gate

Python only, 3.11 or newer. Installs from git today (0.1.0).

Next: a 4-week pilot with a real team



Before

Agree success criteria; baseline from past pull requests

Week 1

pcat init, audit, first patterns from the team's own code

Week 2

Normal work with Claude Code; checks required

Week 3

Fix patterns agents missed or misused

Week 4

Measure, interview, write up

It measures: how often agents fetch the pattern; whether first-time code matches it; that every high deviation gets a human decision; false CI failures per week; time to write a pattern; review effort (a hypothesis).

Appendix: method and caveats

Study	Setup	Size	Main caveat
Variance (v2.1)	Same request, with and without a reference	480 samples, 8 tasks	Lab prompts
Real apps (v5)	3 open-source apps as stand-in orgs	144 runs	4 tasks carry the effect
Gate (v4.1, v5)	Lab requests under the v4.1 rules	60 runs	Requests asked for it
Linter (v5.1)	Real-app runs re-scored, no new runs	144 runs	No general error rate
Neighbour (v5.2)	Flawed function planted next to work	108 runs	6 runs carry the effect

Everywhere: one model (claude-sonnet-5-5 through Claude Code); catalogs picked and reviewed by the builder, not by app maintainers; costs are the CLI's own estimates. Sources and caveats for every number: docs/pitch/claims.md.